BookTrendsRecSys — Technical Report

Team BookTrends

October 16, 2025

Abstract

BookTrendsRecSys is a compact recommendation system that produces a Top–10 list tailored to a reader. The end-to-end pipeline spans *data collection* (Goodreads reviews → user rating histories), *data preparation* and EDA, an *implicit-feedback ALS* model, *offline evaluation* with ranking metrics (P@10, R@10, NDCG@10), a descriptive *fairness snapshot*, and a *Streamlit* app. Final test metrics: P@10=**0.0240%**, R@10=**0.0204%**, NDCG@10=**0.0249%**; validation NDCG@10=**0.0148%**. Parity gaps: region=**0.0052 pp**, genre=**0.0786 pp**. We emphasize strict split hygiene, resumable crawling, and reproducibility.

1 Introduction

We optimize for short-list quality rather than rating error: P@10 (share of good picks among 10), R@10 (coverage within 10), NDCG@10 (order-aware list quality). Hyperparameters are selected on validation NDCG@10; seed is 42.

2 Data Collection and Crawling Strategy

Layout & scripts. `/scripts`

```
get_rating_of_users.py   # Fetch rating lists for users from review files
get_rating_of_books.py   # Fetch ratings for books based on a list of book_ids
build_goodreads_dataset.py   # Merge all rating files into a unified dataset
count_books.py   # Count the number of books reviewed by users
goodreads_reviews__book_id.json   # Original review files containing user_id
goodreads_ratings_{user_id}.json   # Rating results for individual users
goodreads_dataset.csv   # Merged training dataset (userid, bookid, rating)
```

Workflow. Start from popular book lists (seeds) → scrape book review pages to extract `user_id`s → visit each user's rating history (if public) → aggregate to `(userid, bookid, rating)` with timestamps and shelves when available.

Parsing & selectors. Robust CSS queries with fallbacks: review rows via `tr.bookalike.review` (fallback `table#books tr`); ratings via `.ShelfStatus span[aria-label*="out of 5"]` to extract star counts; titles/IDs from title cells else cover cells/anchors. Handle alternate layouts.

Network hygiene. Use a `requests.Session` with `HTTPAdapter`+Retry for 429/5xx; randomized delays; cookies and headers; pagination until no new rows or a safe cap. Resume-safe: skip existing JSON unless overwriting; validate after write.

Schemas. Book review JSON: `{book_id, book_title, average_rating, reviews:[{user, user_id, user_rating, review_date, review, likes, comments}], fetched_at}`. User ratings JSON: `{source_user_id, count, items:[{book_id, book_title, book_url, author, user_rating_text, user_rating, date_read, date_added, shelves, review_url}]}`. Ethics/ToS: public data only, rate-limit, minimal PII.

3 Dataset & EDA

Inputs in `data/raw/`; derived artifacts in `data/processed/`; figures in `figs/eda/`. Summary:

Table 1: Dataset summary (N/A if not available).

Books	**10,000**
Users	**53,424**
Interactions	**5,976,479**
Sparsity	**98.88%**
Year span	**1513–2017**
Top languages	**eng (6,341), en-US (2,070), nan (1,084), en-GB (257), ara (64)**

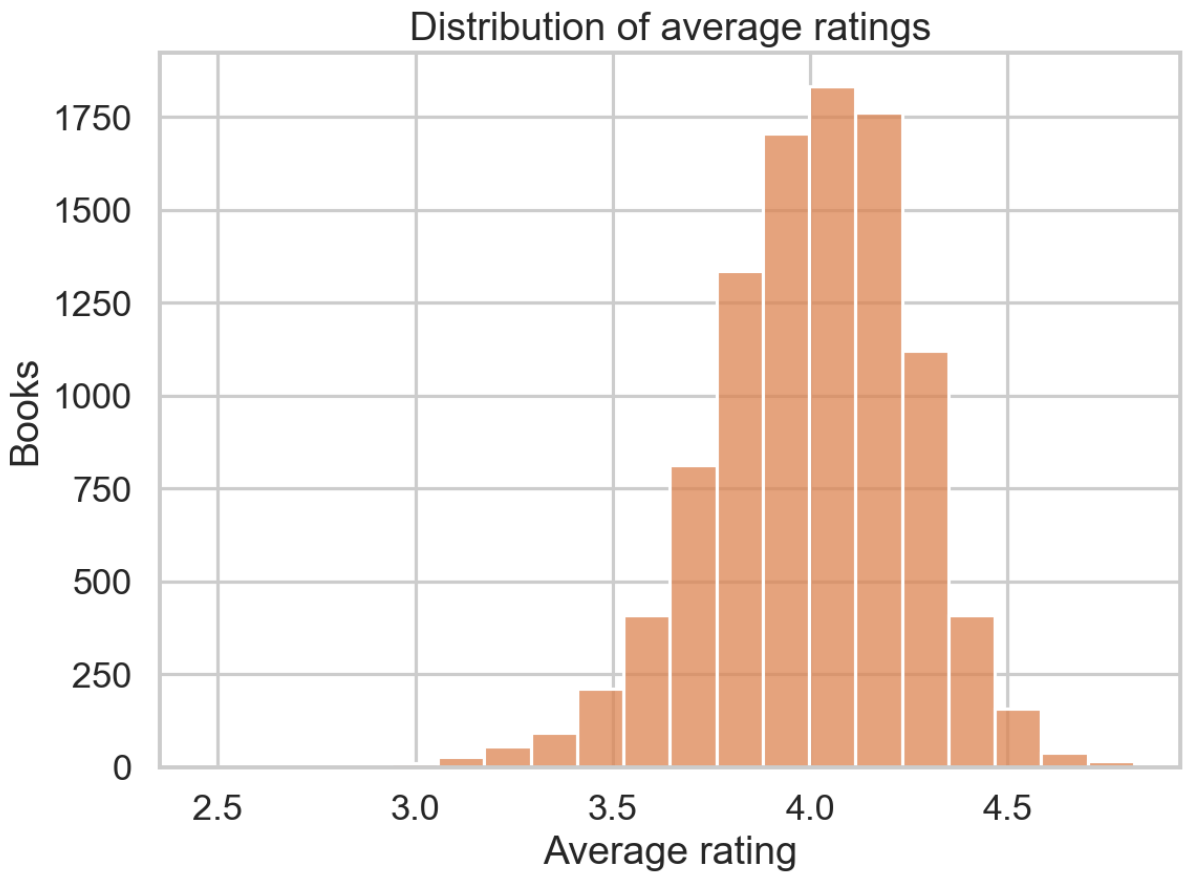

Figure 1: Average rating distribution (catalog).

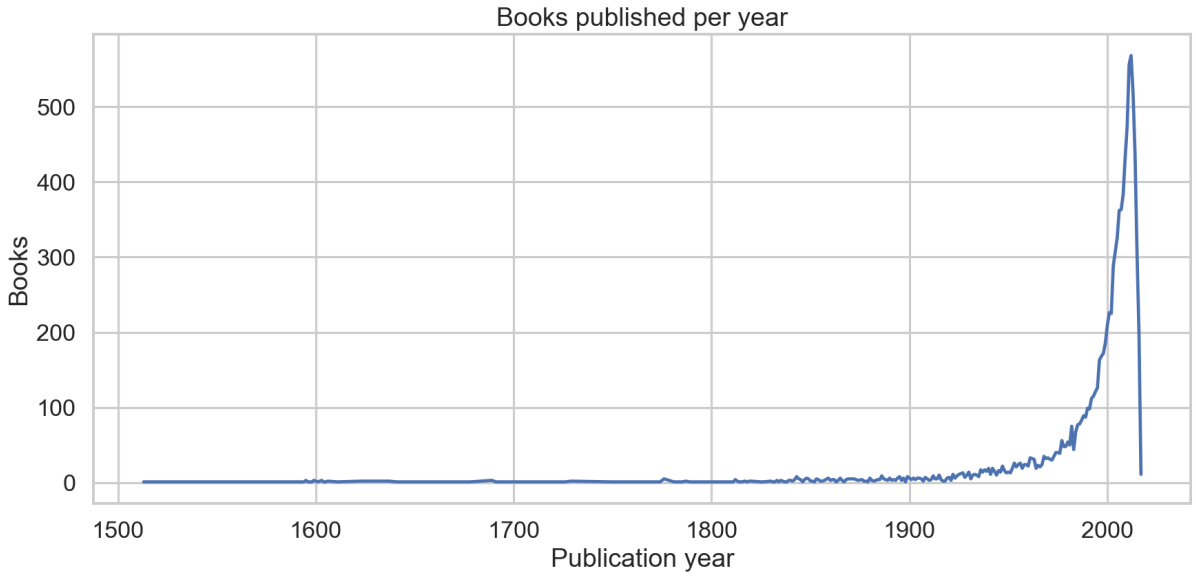

Figure 2: Catalog growth: books per year.

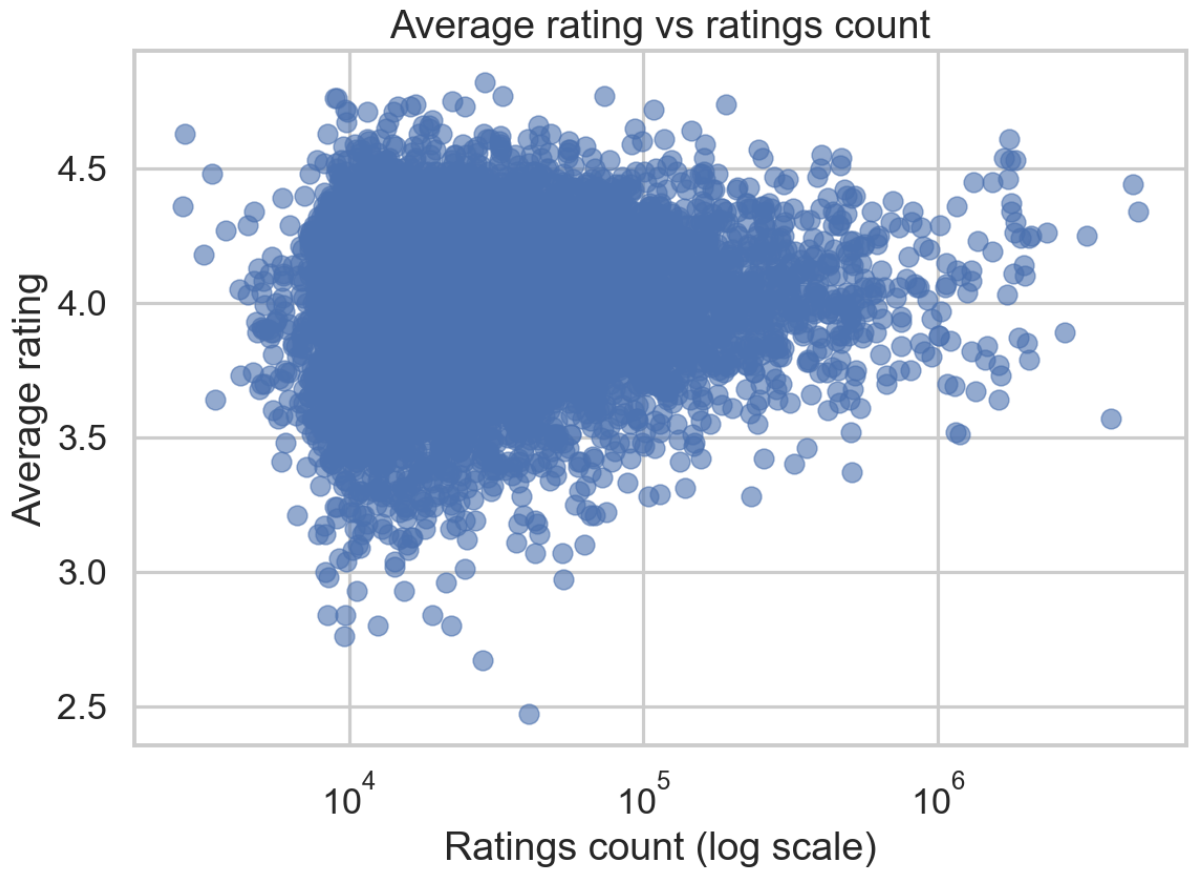

Figure 3: Rating vs. popularity (ratings count, log scale).

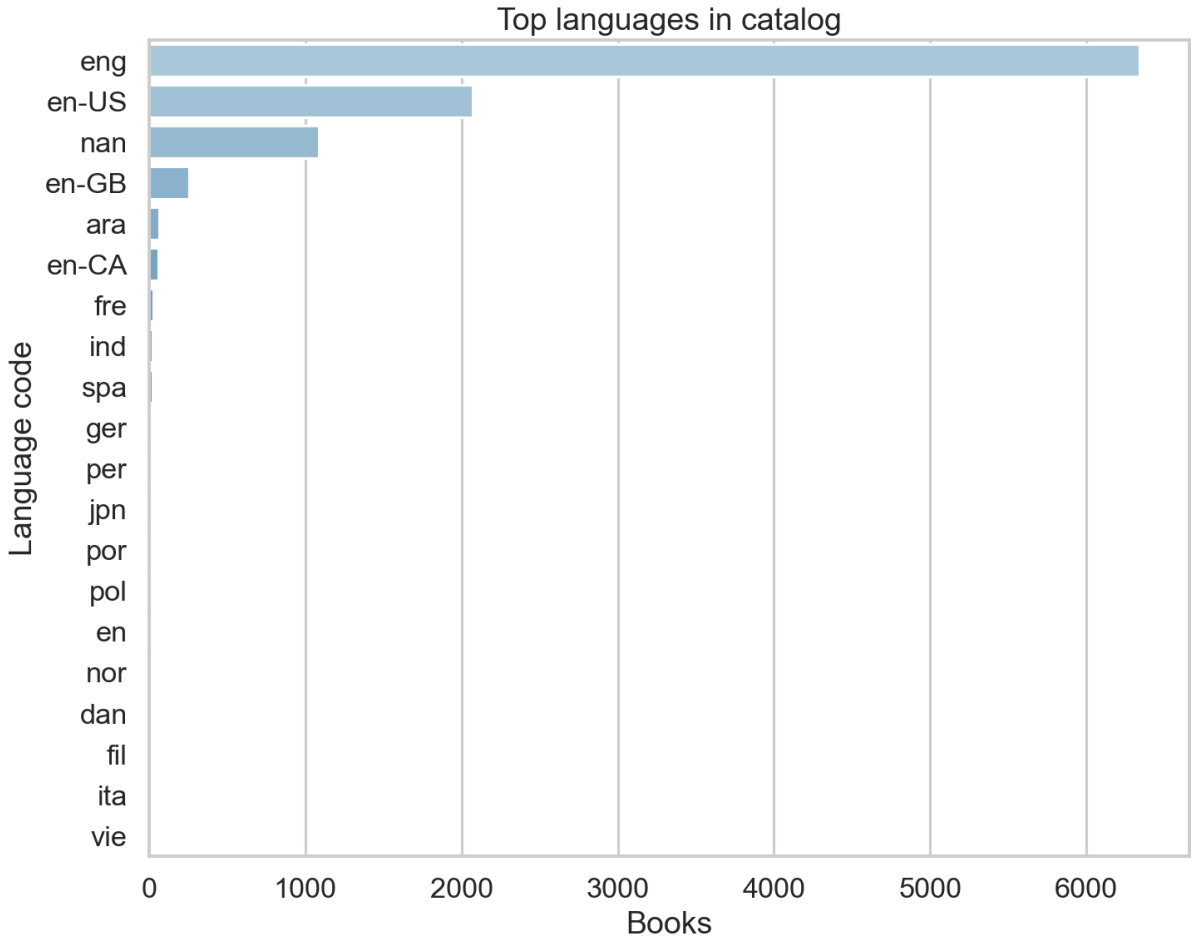

Figure 4: Top languages in the catalog.

4 Model: Implicit ALS

We use Hu–Koren–Volinsky implicit ALS. With preference $p_{ui} \in \{0, 1\}$ and confidence $c_{ui} = 1 + \alpha r_{ui}$, we solve:

$$\min_{X,Y} \sum_{u,i} c_{ui} \left(p_{ui} - x_u^\top y_i\right)^2 + \lambda(\|x_u\|^2 + \|y_i\|^2),$$

via alternating least-squares. Chosen hyperparameters: rank=**64**, reg=**0.05**, alpha=**20**, iters=**10**.

5 Evaluation

For user u with ground truth G_u and top-K list $R_u = [i_1, \ldots, i_K]$ (K=10):

$$\text{P@K} = \frac{1}{|U|} \sum_u \frac{|R_u \cap G_u|}{K}, \quad \text{R@K} = \frac{1}{|U|} \sum_u \frac{|R_u \cap G_u|}{|G_u|},$$

$$\text{DCG@K}(u) = \sum_{j=1}^{K} \frac{\mathbf{1}[i_j \in G_u]}{\log_2(j+1)}, \quad \text{NDCG@K} = \frac{1}{|U|} \sum_u \frac{\text{DCG@K}(u)}{\text{IDCG@K}(u)}.$$

Table 2: Final offline metrics (higher is better).

Precision@10	**0.0240%**
Recall@10	**0.0204%**
NDCG@10	**0.0249%**
Validation NDCG@10	**0.0148%**

6 Fairness Snapshot (descriptive)

We compute NDCG@10 by group and report gap $\max_g - \min_g$. Region gap: **0.0052 pp**; genre gap: **0.0786 pp**. Many genre slices are sparse; treat as descriptive signals, not causal claims.

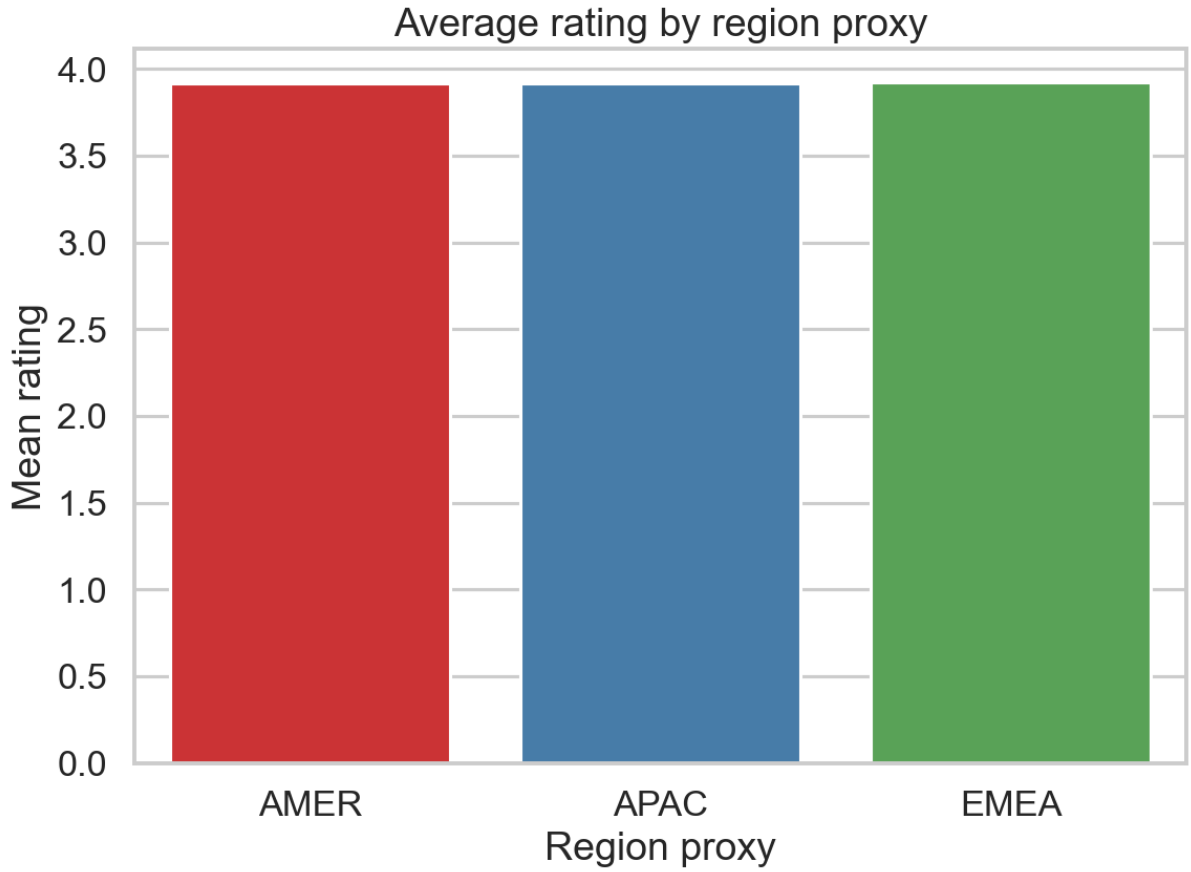

Figure 5: Region snapshot (NDCG@10).

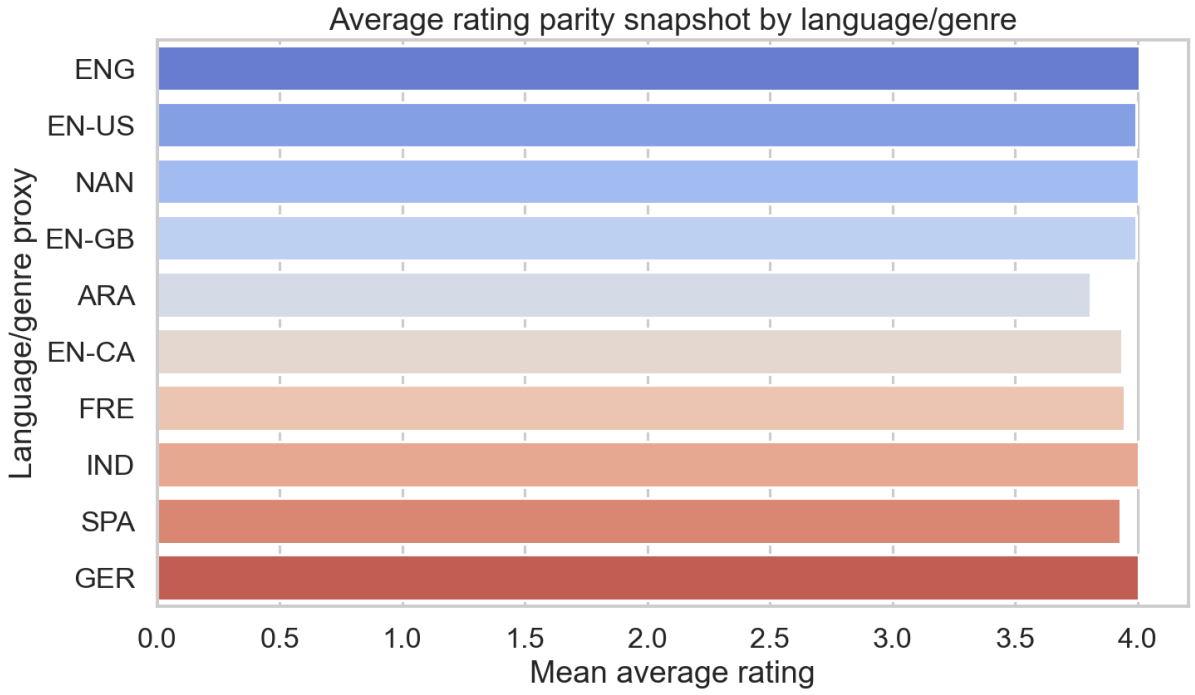

Figure 6: Language/genre proxy snapshot (NDCG@10).

7 Application & Serving

Streamlit app (`app/app.py`) with *Recommendations* (Top–10 by user/genre) and *Insights* (KPI + EDA charts). Uses precomputed factors/metrics; add popularity fallback for cold start.

1. `pip install -r requirements.txt`
2. `make data && make train && make eval && make parity`
3. `streamlit run app/app.py`

8 Reproducibility

Seed=42; strict train/val/test. Artifacts: `models/`, `metrics/`, `figs/eda/`, `data/processed/`. Compile this report: `cd deliverables/2025-10-16-spotlight/report`
`latexmk -pdf report.tex`

Appendix: Worked NDCG Example

For K=5, $G_u = \{b2, b5, b7\}$ and $R_u = [b1, b5, b9, b2, b8]$: hits at ranks 2,4. P@5=2/5 = 40%; R@5=2/3 $\approx$ 66.7%. DCG@5 $\approx$ 1.062, IDCG@5 $\approx$ 2.131, NDCG@5$\approx$ 49.8%.

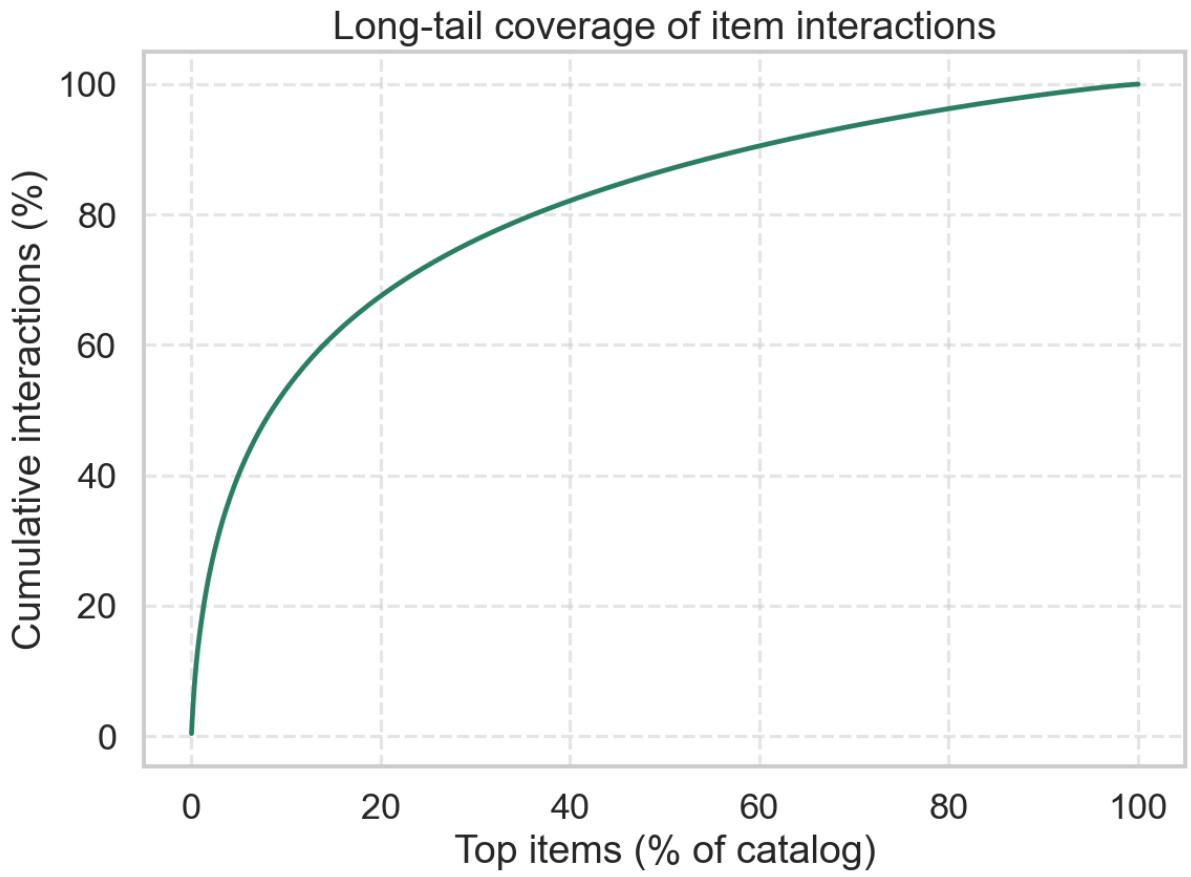

Figure 7: Long tail: small set of items receives most interactions.